

把上述程序保存成 `test.sh` 后，再执行 `chmod +x test.sh`，即可用 `./test.sh` 来执行它。当然，扩展名也不是必需的，完全可以以不带扩展名的 `test` 命名。

上面的程序不是最简洁的（例如，可以直接把 `diff` 命令放在 `if` 语句中），但展示了 `bash` 脚本的一些其他用法。例如，`while` 循环是“`while <命令集>; do <命令集>; done`”，而 `if` 语句的基本是“`if <命令集>; do <命令集>;`”。不管是 `while` 还是 `if`，判断的都是命令集中最后一条语句的返回码（`exit code`）是否为 0。例如，若把上面的脚本改成 `if diff output.a output.b; then break; fi`，则当两个文件相同（`diff` 返回码为 0）时退出循环（这个不是我们所期望的）。如果忘记了命令格式，可以用 `help if` 和 `help while` 获取帮助。

上面的“`true`”和“`[`”都是程序。前者的作用是直接返回 0；而后者的作用是计算表达式（该程序要求最后一个参数必须是“`]`”），其中“`$?`”是 `bash` 内部变量，表示“上一个程序的返回码”。

### A.2.3 再谈随机数

如果做过测试，可能会发现上面的方法有一个问题：如果程序执行太快，随机数生成器在相邻两次执行时，`time(NULL)` 函数返回值相同，因而产生出完全相同的输入文件。换句话说，每隔一秒才能产生出一个不同的随机数据。一个解决方案是利用系统自带的随机数发生器：在 Windows 下是环境变量 `%random%`，而在 `bash` 中是 `$RANDOM`。它们都是 0~32767 之间的随机整数。可以直接用脚本编写随机数生成器，也可以把它们传递到程序中。

## A.3 编译器和调试器

既然编译器和调试器都是程序，执行方法和普通程序大致相同。在安装时，系统会自动把编译器和调试器程序所在路径加到搜索路径中，因此在执行时不必像 `./gcc` 这样加上路径名。

### A.3.1 gcc 的安装和测试

尽管在现场比赛中，编译器都已安装好，但如果平时练习，一般需要自己安装。如果使用 Linux，在安装操作系统时即可选择安装 `gcc`、`g++`、`binutils` 等包，但若要在 Windows 中使用 C/C++ 语言，需要手工安装编译器。

本书推荐使用 MinGW 环境下的 `gcc`，它的好处是和 Linux 下的 `gcc` 一致性较好，而且是免费的。可以到 [www.mingw.com](http://www.mingw.com) 中下载最新的安装包，然后在安装时选择 `g++` 编译器。

安装完毕后，在命令行中执行 `gcc` 命令。如果显示 `gcc: no input files`，则安装成功；如果提示不存在这个命令，可能是因为没有把 `gcc` 所在目录加到搜索路径中。可以双击控制面板的“系统”图标，并在“高级”选项卡中设置环境变量。在“系统变量”中找到“`PATH`”（大小写无所谓），它就是可执行程序的搜索路径。请在它的最后加入 MinGW 安装路径的

bin 子目录，如 C:\MinGW\bin（在安装时记住 MinGW 的安装路径），保存后重新启动命令行，gcc 就应该可以正常工作了。

### A.3.2 常见编译选项

先建立一个 test.c，试试常见的编译选项。

```
#include<stdio.h>
main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    int c = a+b;
    printf("%d%d\n", c);
}
```

编译一下，命令为 gcc test.c。程序没有输出，代表一切均好。检查目录（Windows 下用 dir，Linux 下用 ls），会发现多了一个 a.exe（Windows）或 a.out（Linux），这就是程序的编译结果。

gcc test.c-o test 命令会让编译出的可执行程序名为 test.exe（Windows）或 test（Linux）。这样，就能用 test（Windows）或 ./test（Linux）方式运行程序。

也许读者已经看出了上述代码中的一些问题，不过当程序更加复杂时，人眼就不一定能快速找到错误了。在这样的情况下，编译选项能起作用：gcc -test.c -o test -Wall。这次，编译器指出了 3 个警告：main 函数没有返回类型、没有返回值、printf 的格式字符串可能有问题。还可以进一步用 -ansi-pedantic，它会检查代码是否符合 ANSI 标准（-ansi 只是判断是否和 ANSI 冲突，而 -pedantic 更加严格）。它进一步指出了上述代码中的另一个问题：ANSI C 中不允许临时声明变量，而必须在语句块的首部声明变量。

在 C 语言中，另一个常用的编译选项是 -lm，它让编译器连接数学库，从而允许程序使用 math.h 中的数学函数。C++ 编译器会自动连接数学库，但如果程序的扩展名是 .c，且不连接数学库，有时会出现意想不到的结果。

另一个有用的选项是 -DDEBUG，它在编译时定义符号 DEBUG（可以换成其他，如 -DLOCAL 将定义符号 LOCAL），这样，位于 #ifdef DEBUG 和 #endif 中间的语句会被编译。而在通常情况下，这些语句将被编译器忽略（注意，不仅是不会执行，连编译都没有进行）。

可以用 -O1、-O2 和 -O3 对代码进行速度优化。一般情况下，直接编译出的程序比用 -O1 编译出的程序慢，而后者比 -O2 慢。尽管理论上 -O3 编译出的程序更快，但由于某些优化可能会误解程序员的意思，一般比赛中不推荐使用。另外，如果你的程序中有一些不确定因素（如使用了未初始化的变量），运行结果可能会和编译选项有关——用 -O1 和 -O2 编译出的程序也许不仅是速度有差异，答案甚至都有可能不同！当然，这种情况出现的前提是程序有瑕疵。如果是一个规范的程序，运行结果不会和优化方式有关。

既然编译选项可以影响程序的行为，在正规比赛中，组织方应提前公布编译选项。如果没有公布，选手最好尽早询问。

### A.3.3 gdb 简介

`gdb` 尽管只是一个文本界面的调试器，但功能十分强大。不管是 Linux 和 Windows 下的 MinGW，`gcc` 和 `gdb` 都是最佳拍档。

`gdb` 的使用方法很简单——用 `gcc` 编译成 `test.exe` 之后，执行 `gdb test.exe` 即可。不过，如果要用 `gdb` 调试，编译时应加上 `-g` 选项，生成调试用的符号表。

接下来使用 `l` 命令，将看到部分源程序清单。如果用 `l 15`，将会显示第 15 行（以及它前后的若干行）。除此之外，还可以用函数名来定义，如 `l main` 将显示 `main` 函数开头的附近 10 行。如果不加参数执行 `l`，将显示下 10 行；`list -` 将显示上 10 行。所有这些操作都可以用 `help list` 命令来查看。`gdb` 中的命令可以简写（例如 `list` 简写成 `l`），大家可以多尝试（提示：试一下命令的前若干个字母）。

运行程序的命令是 `r` (`run`)，但会一直执行到程序结束。如何让它停下来呢？方法是用 `b` (`break`) 命令设置断点。例如，`b main` 命令将在 `main` 函数的开始处设置一个断点，则用 `r` 命令执行时会在这里停下来。如果想继续运行，请用 `c` (`continue`) 命令，而不是继续用 `r` 命令。和 `list` 命令类似，`b` 命令既可以指定行号，也可以在指定函数的首部停下来。笔者在调试很多程序时都是以命令 `b main` 和 `r` 开头的。

如果希望逐条语句地执行程序，不停地用 `b` 和 `c` 命令太麻烦。`gdb` 提供了一些更加方便的指令，其中最常用的有两个：`next`（简写为 `n`）和 `step`（简写为 `s`）。其作用都是执行当前行，区别在于如果当前行涉及函数调用，则 `next` 是把它作为一个整体执行完毕，而 `step` 是进入函数内部。尽管 `n` 和 `s` 都只有一个字母，但有时还是稍显繁琐。在 `gdb` 中，如果在提示符下直接按 `Enter` 键，等价于再次执行上一条指令，因此如果需要连续执行 `s` 或者 `n`，只需要第一次输入该命令，然后直接连接 `Enter` 键即可。另外，和命令行一样，可以按上下箭头来使用历史记录。

另一个常用命令是 `until`（简写为 `u`），让程序执行到指定位置。例如，`u 9` 就是执行到第 9 行，`u doit` 就是执行到 `doit` 函数的开头位置。

停下来以后便打印一些函数值，看看是否和想象的一致。用 `p` (`print`) 命令可以打印出一些变量的值，而 `info locals`（可以简写为 `i lo`）可以显示所有局部变量。如果希望每次程序停下来，则可以用 `display`（简写为 `disp`）命令。例如，`display i+1` 就可以方便地读取 `i+1` 的值。它往往和 `n`、`s` 和 `u` 等单步执行指令配合使用。如果需要列出所有 `display`，可以用 `info display`（简写为 `i disp`）；还可以删除或者临时禁止/恢复一些 `display`，相应的命令为 `delete display` (`d disp`)、`disable display` (`dis disp`) 和 `enable display` (`en disp`)。类似地，也可以根据断点编号删除、禁止和恢复断点，还可以用 `clear` (`cl`) 命令，像 `b` 命令一样根据行号或者函数名直接删除断点。

在多数情况下，灵活运用上述功能已经能高效地调试程序了。下面把涉及的命令列出，供读者参考，如附表 A-2 所示。

附表 A-2 gdb 常见命令

| 简 写  | 全 称      | 备 注                                                   |
|------|----------|-------------------------------------------------------|
| l    | list     | 显示指定行号或者指定函数附近的源代码                                    |
| b    | break    | 在指定行号或者指定函数开头处设置断点。如 b main                           |
| r    | run      | 运行程序，直到程序结束或者遇到断点而停下                                  |
| c    | continue | 在程序中断后继续执行程序，直到程序结束或者遇到断点而停下。注意在程序开始执行前只能用 r，不能用 c    |
| n    | next     | 执行一条语句。如果有函数调用，则把它作为一个整体                              |
| s    | step     | 执行一条语句。如果有函数调用，则进入函数内部                                |
| u    | until    | 执行到指定行号或者指定函数的开头                                      |
| p    | print    | 显示变量或表达式的值                                            |
| disp | display  | 把一个表达式设置为 display，当程序每次停下来时都会显示其值                     |
| cl   | clear    | 取消断点，和 b 的格式相同。如果该位置有多个断点，将同时取消                       |
| i    | info     | 显示各种信息。如 i b 显示所有断点，i disp 显示 display，而 i lo 显示所有局部变量 |

如果对上述解释有疑问，可输入 help 以获得详尽的帮助信息。

### A.3.4 gdb 的高级功能

gdb 的功能远不止刚才所讲述的那些。尽管很多功能是专为系统级调试所设，但还有很多功能也能为算法程序的调试带来很大方便。

首先是栈帧的相关命令，其中最常用的是 bt，其他命令可以通过 help stack 来学习。接下来是断点控制命令。commands (comm) 命令可以指定在某个断点处停下来后所执行的 gdb 命令，ignore (ig) 命令可以让断点在前 count 次到达时都不停下来，而 condition 则可以给断点加一个条件。例如，在下面的循环中：

```
10 for(i = 0; i < n; i++)
11 printf("%d\n", i);
```

首先用 b 11 设置断点（假设编号为 2），然后用 cond 2 i==5 让该断点仅当 i=5 时有效。这样的条件断点在进行细致的调试时往往很有用。

另外，gdb 还支持一种特殊的断点——watchpoint。例如，watch a（简写为 wa a）可以在变量 a 修改时停下，并显示出修改前后的变量值，而 awatch a（简写为 aw a）则是在变量被读写时都会停下来。类似地，rwatch a（rw a）则是在变量被读时停下。

最后需要说明的是，gdb 中可以自由调用函数（不管是源程序中新定义的函数还是库函数）。第一种方法是用 call 命令。例如，如果想给包含 10 个元素的数组 a 排序，可以像这样直接调用 STL 中的排序函数 call sort(a, a+10)。

遗憾的是，如果真的做过这个实验，会发现刚才所说完全是骗人的。gdb 会显示不存在函数 sort。怎么会这样呢？如果学过宏和内联函数就会知道，很多看起来是函数的却不一定是函数，或者说，不一定是调试器识别的函数。为了在 gdb 中调用 sort，可以将它打包：